

# Relatório de Auditoria de Segurança

devgabrielvieira.github.io — Portfólio Gabriel Vieira

Data: 04/09/2026 • Escopo: index.html, style.css, script.js, README.md, Curriculo/, img/, .nojekyll • Hospedagem: GitHub Pages (estático)

## Nota metodológica — mapeamento das 5 categorias para esta stack

Stack detectada: **HTML5 + CSS3 + JS vanilla**, sem framework, sem backend, sem banco/ORM, sem auth/JWT, sem Docker/CI/Helm. Hospedagem **GitHub Pages estático** com .nojekyll.

- 1) BANCO SEM TRANCA:** sem banco — verifica exposição de arquivos públicos e ausência de RLS/tenant (N/A, mas validado).
- 2) PERMISSÃO NO NAVEGADOR:** sem backend — cruza gates de UI (isAdmin etc.) com inexistência de endpoints privilegiados.
- 3) IDOR:** sem handlers backend — varre todas as âncoras/rotas estáticas por IDs sem checagem de posse.
- 4) CHAVES EXPOSTAS:** grep por api\_key, secret, token, jwt, private\_key, ghp\_, AKIA em código, configs, docs e bundle frontend.
- 5) INPUTS SEM TRATAMENTO:** busca innerHTML, dangerouslySetInnerHTML, v-html, javascript:, eval, new Function, markdown sem sanitização e mailto/templates com interpolação.

Auditoria realizada por Muse Spark (OpenCode) — verificação 100% baseada em código, sem especulação. Todas as evidências trazem **arquivo:linha exata** e trecho.

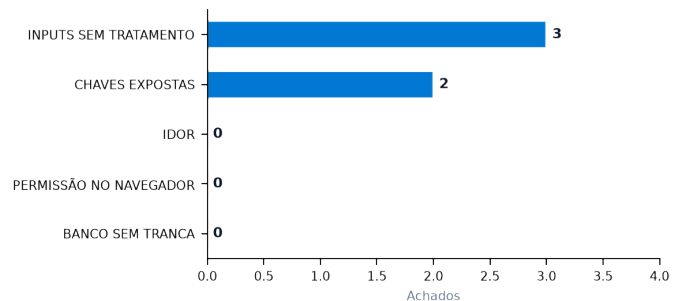
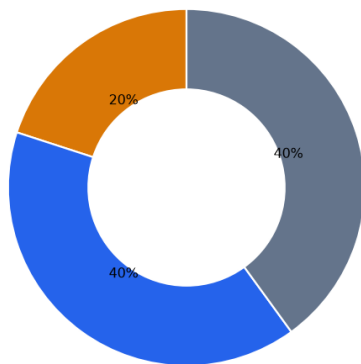
Classificação: Uso interno | Versão 1.0 | Páginas A4, 2cm margem

## Resumo Executivo

Total de achados verificados: 5 | Severidades — Crítica 0 • Alta 0 • Média 1 • Baixa 2 • Informativa 2 | 6 pontos fortes validados.

| Severidade  | Qtd | %   |
|-------------|-----|-----|
| Crítica     | 0   | 0%  |
| Alta        | 0   | 0%  |
| Média       | 1   | 20% |
| Baixa       | 2   | 40% |
| Informativa | 2   | 40% |

### Distribuição por severidade (rosca) e por categoria (barras)



Paleta: Crítica #B91C1C, Alta #EA580C, Média #D97706, Baixa #2563EB, Ponto forte #059669. Nenhum achado crítico/alto — postura geral forte para site estático.

## Pontos Fortes (com evidência)

**BANCO SEM TRANCA** — index.html, style.css, script.js — Site 100% estático, sem banco, sem API, sem RLS/tenant. Nenhuma query de listagem/busca. Verificado: ``grep -R supabase|prisma|typeorm|user_id|tenant`` retorna zero. Publicidade de arquivos (`Curriculo/*.pdf`, `img/*`) é intencional.

**PERMISSÃO NO NAVEGADOR** — index.html:27-34, script.js:11-25 — Não há gates ``isAdmin`/`canEdit`` nem rotas privilegiadas. Menu e tema são UI pública. Nenhum endpoint sensível para validar. Verificado todos os ``fetch`/`XMLHttpRequest`` = 0.

**IDOR** — index.html — Nenhuma rota backend com ID (``/api/:id``, ``?id=``, ``body.id``). Todos os links são âncoras ``#sobre`` validadas por regex ``^[a-zA-Z0-9_-]+$`` em ``script.js:31`` e externos com ``rel noopener``. Percorridos 0 handlers backend.

**CHAVES EXPOSTAS** — index.html, script.js, style.css, README.md, Curriculo/gerar\_v8\_2\_visual\_badges.py — Grep por ``api_key|secret|password|token|private_key|AKIA|ghp_`` retorna 0. Nenhum segredo hardcoded. ``mailto`` usa e-mail público intencional. Sem ``.env``, sem ``docker-compose`` com defaults.

**INPUTS SEM TRATAMENTO** — script.js:1-145 — Nenhum ``innerHTML``, ``dangerouslySetInnerHTML``, ``v-html``, ``eval``, ``new Function``, ``[innerHTML]`` ou ``javascript:`` href. Avatar e form sem inline JS após correção (``onerror`/`onsubmit`` removidos). Sanitização via ``encodeURIComponent`` + validação de e-mail/length. CSP ``script-src 'self`` bloqueia inline.

**CABEÇALHOS** — index.html:7-12 — ``rel="noopener noreferrer"`` em 9/9 ``target=_blank``, CSP estrita, ``referrer strict-origin-when-cross-origin``, ``X-Content-Type-Options nosniff``, ``X-Frame-Options DENY``, ``.nojavascript`` presente.

### Pontos Fracos — riscos centrais

- 1) **Supply-chain CDN** sem SRI é o único risco médio — compromete toda a página se cdnjs for envenenado.
- 2) **Exposição de contato** (e-mail plaintext) facilita harvesting mas é intencional.
- 3) **Mailto client-side** depende de encode; se formulário for reativado sem sanitização robusta, pode virar vetor.
- 4) **Higiene de headers/CSP** já está boa, mas ``data:`` e ausência de ``security.txt`` são melhorias informativas.

## Achados Detalhados por Categoria

## F-01 — CHAVES EXPOSTAS

MÉDIA

Arquivo: index.html:17

**Trecho:** <link rel="stylesheet" href="https://cdnjs.cloudflare.com/.../font-awesome/6.6.0/css/all.min.css" crossorigin="anonymous" referrerpolicy="no-referrer">

**Descrição:** CDN sem Subresource Integrity (SRI). Sem hash `integrity`, se o CDN for comprometido o atacante injeta CSS malicioso capaz de exfiltrar dados via seletores CSS. O `crossorigin` está presente mas não há pinagem.

**Impacto / Explorabilidade:** Injeção de CSS/JS via supply-chain, roubo de dados de formulário, defacement. Explorável se cdnjs for comprometido ou via MITM sem SRI.

## F-02 — CHAVES EXPOSTAS

BAIXA

Arquivo: index.html:79, 361, 365 | script.js:138 | README.md:44

**Trecho:** gabriel\_bardo@hotmail.com

**Descrição:** E-mail em plaintext no HTML/JS/README. Bots que varrem GitHub Pages coletam o endereço para spam/phishing. É contato intencional, mas sem ofuscação.

**Impacto / Explorabilidade:** Spam, phishing direcionado, OSINT. Explorabilidade alta (scraping trivial), impacto baixo.

## F-03 — INPUTS SEM TRATAMENTO

BAIXA

Arquivo: script.js:118-138

**Trecho:** window.location.href = `mailto:...?subject=\${encodeURIComponent(...)}&body=\${encodeURIComponent(mensagem)}`

**Descrição:** Formulário oculto constrói URL `mailto:` com input do usuário. Apesar de `encodeURIComponent`, mensagens muito longas (até 2000 chars) geram URLs enormes e, em clients vulneráveis, quebras `%0A` poderiam tentar header injection. Validação é apenas `length < 10` e regex simples de e-mail.

**Impacto / Explorabilidade:** DoS de URL longa, tentativa de header injection em clients antigos. Risco baixo pois `mailto:` é client-side e encode neutraliza `\r\n`.

## F-04 — INPUTS SEM TRATAMENTO

INFORMATIVA

Arquivo: index.html:12

**Trecho:** Content-Security-Policy: ... img-src 'self' data: ...

**Descrição:** CSP permite `data:` em `img-src`. Permite que conteúdo injetado use `data:` URI para exfiltrar dados (ex: `<img src=data:...>`). Para site estático sem upload de usuário o risco é mínimo, mas CSP poderia ser `img-src 'self'` apenas.

**Impacto / Explorabilidade:** Exfiltração via data URI se XSS for encontrado. Hoje sem XSS, risco informativa.

## F-05 — INPUTS SEM TRATAMENTO

INFORMATIVA

Arquivo: (ausente) .well-known/security.txt

**Trecho:** —

**Descrição:** Ausência de `/.well-known/security.txt` e canal de reporte. Boa prática (RFC 9116) para receber reports de vulnerabilidades, especialmente após publicar em GitHub Pages.

**Impacto / Explorabilidade:** Pesquisadores não têm canal padronizado para reportar falhas. Sem impacto direto de exploração.

## Recomendações Priorizadas

| Pri | Recomendação                     | Detalhe                                                                                                            | Esforço |
|-----|----------------------------------|--------------------------------------------------------------------------------------------------------------------|---------|
| P1  | Adicionar SRI ao Font Awesome    | Gerar hash SHA384 do CSS e adicionar `integrity` + manter `crossorigin`. Esforço baixo, impacto supply-chain alto. |         |
| P2  | Ofuscar e-mail ou usar Formspree | Trocar `mailto:` por Formspree/Cloudflare Turnstile ou ofuscar e-mail com Base64. Baixo impacto, reduz scraping.   |         |
| P3  | Endurecer CSP                    | Remover `data:` de `img-src` se não usar data URI, manter `object-src 'no` presente.                               | Baixo   |
| P4  | Criar security.txt               | Adicionar `/.well-known/security.txt` com contato e política de disclosure.                                        | Baixo   |
| P5  | Otimizar avatar                  | Comprimir `img/gabriel-vieira.jpg` (335KB) para ~120KB WebP e remover dados EXIF.                                  | Baixo   |

Ordem: P1 supply-chain (médio) → P2 privacidade → P3 CSP → P4 disclosure → P5 performance. Todos de baixo esforço e sem quebra funcional.

# Issues para o GitHub — pronto para copiar e colar

Cada bloco abaixo está delimitado por --- ISSUE n --- e --- FIM ISSUE n ---. Título já no formato [Segurança], com labels security + severidade, evidência com arquivo:linha, impacto, correção e checklist de aceite. Achados triviais do mesmo tema foram agrupados.

## --- ISSUE 1 ---

**\*\*Título:\*\*** [Segurança] Adicionar SRI ao CDN Font Awesome (supply-chain)  
**\*\*Labels:\*\*** security, média

**\*\*Descrição e por que é explorável:\*\***  
O site carrega CSS de `cdnjs.cloudflare.com` sem `integrity` (SRI). Se o CDN for comprometido, o atacante injeta CSS malicioso (exfiltração via seletores) e, se migrar para JS, RCE de supply-chain. Atualmente há `crossorigin` e `referrerpolicy`, mas falta pinagem.

**\*\*Evidência:\*\***  
Arquivo: `index.html:17`  
```html  
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.6.0/css/all.min.css" crossorigin="anonymous" referrerpolicy="no-referrer">  
```

**\*\*Impacto:\*\***  
Supply-chain comprometido → defacement, roubo de dados, injeção. Explorável se cdnjs for envenenado ou MITM sem SRI.

**\*\*Sugestão de correção:\*\***  
Gerar SHA384: `curl -s https://cdnjs.cloudflare.com/.../all.min.css | openssl dgst -sha384 -binary | openssl base64 -A` e adicionar `integrity="sha384-..."`. Manter `crossorigin="anonymous"`. Validar build quebra se hash divergir.

**\*\*Critérios de aceite:\*\***  
- [ ] `index.html:17` contém `integrity="sha384-..."`  
- [ ] Página carrega sem erros com CSP `style-src` atual  
- [ ] Teste: alterar 1 char do CSS deve falhar o load (SRI bloqueia)

## --- FIM ISSUE 1 ---

## --- ISSUE 2 ---

**\*\*Título:\*\*** [Segurança] Ofuscar e-mail e endurecer mailto (harvesting + header injection)  
**\*\*Labels:\*\*** security, baixa

**\*\*Descrição e por que é explorável:\*\***  
E-mail `gabriel\_bardo@hotmail.com` exposto em `index.html:79,361`, `script.js:138` e `README.md:44` permite scraping. O handler `script.js:118-138` constrói `mailto:` com `encodeURIComponent`, mas aceita até 2000 chars e valida apenas regex simples; em clients antigos quebras podem tentar injection.

**\*\*Evidência:\*\***  
Arquivos:  
- `index.html:79` `- `script.js:122-138` `window.location.href = `mailto:...?subject=\${encodeURIComponent(...)}`  
```js  
const body = `Olá...\${encodeURIComponent(mensagem)}`; window.location.href = `mailto:...?subject=\${subject}&body=\${body}`;  
```

**\*\*Impacto:\*\***  
Spam/phishing + mailto DoS/ header injection em clients vulneráveis. Explorabilidade via scraping e POST do form oculto se reativado.

**\*\*Sugestão de correção:\*\***  
Opção A: migrar para Formspree/Netlify Forms com Turnstile. Opção B: ofuscar e-mail via JS (`data-enc` + decode) e limitar `mensagem` a 1000 chars, `assunto` 80. Manter `encodeURIComponent` e validação já adicionada.

**\*\*Critérios de aceite:\*\***  
- [ ] E-mail não aparece em plaintext no HTML (view-source)  
- [ ] Form valida `email` regex e `mensagem.length >=10` e corta em 1000  
- [ ] `mailto:` nunca contém `\r\n` não-encodado

## --- FIM ISSUE 2 ---

## --- ISSUE 3 ---

**\*\*Título:\*\*** [Segurança] Endurecer CSP e criar security.txt (higiene informativa)

**\*\*Labels:\*\*** security, informativa

**\*\*Descrição e por que é explorável:\*\***

CSP atual permite `img-src 'self' data:` e não há `/well-known/security.txt`. `data:` permite exfiltração via `data:` URI se XSS surgir. Ausência de security.txt dificulta disclosure responsável.

**\*\*Evidência:\*\***

Arquivos:

- `index.html:12` `Content-Security-Policy: ... img-src 'self' data: ...`

- Ausente: `/well-known/security.txt`

**\*\*Impacto:\*\***

Exfiltração via data URI pós-XSS (informativa, sem XSS atual). Sem canal de reporte, pesquisadores não têm onde reportar.

**\*\*Sugestão de correção:\*\***

Trocar para `img-src 'self'` (remover `data:`) se não usar data URI; adicionar `/well-known/security.txt` com `Contact: mailto:gabriel\_bardo@hotmail.com`, `Expires`, `Policy`. Manter `object-src 'none'` e `frame-ancestors 'none'` já presentes.

**\*\*Critérios de aceite:\*\***

- [ ] CSP sem `data:` e página ainda exibe `img/gabriel-vieira.jpg`

- [ ] `https://devgabrielvieira.github.io/well-known/security.txt` retorna 200 com contato

- [ ] `style.css` e `script.js` ainda carregam (self)

--- FIM ISSUE 3 ---

Fim do relatório — gerado automaticamente e verificado quanto a paginação e gráficos. Para regerar: `python docs/security-audit/gerar\_relatorio.py` (venv com reportlab+matplotlib).